Digital Electronics (ECT-033)

Complete unit-wise notes - easy English, matched to the UTU syllabus.

Study Materials by Anas - study-materials-by-anas.vercel.app

Unit 1 - Minimization of Logic Functions

What this unit covers: simplifying Boolean functions so the final circuit uses fewer gates - K-maps, Quine-McCluskey, and number-system basics.

Key ideas: Boolean algebra rules (De Morgan, absorption, consensus) turn long expressions into short ones. A **Karnaugh map** (K-map) does it visually: place minterms in Gray-code order, then group 1s in powers of 2 (2, 4, 8...) - each group drops the variables that change inside it. Groups can overlap and wrap around the edges; **don't cares** (X) may join groups but never need their own. SOP grouping gives AND-OR; grouping 0s gives POS. **Quine-McCluskey** is the tabular method for 5+ variables: pair minterms differing in one bit, find prime implicants, then cover the minterms with essential prime implicants. Revise number systems too: binary/hex/octal conversion and 1's/2's complement subtraction.

Exam pointers: one 3-variable and one 4-variable K-map minimisation are near-certain - show groups clearly on the map. A small Quine-McCluskey table (given minterm list) and a "convert + complement arithmetic" question round out the unit.

Unit 2 - Combinational Circuits

What this unit covers: combinational circuits - outputs depend only on the present inputs: adders, subtractors, multiplexers, demultiplexers, encoders, decoders, comparators.

Key ideas: Half adder: Sum = A⊕B, Carry = AB. **Full adder**: Sum = A⊕B⊕Cin, Carry = AB + BCin + ACin (two half adders + OR). A **MUX** (2^n:1) picks one of 2^n inputs using n select lines - any Boolean function of n variables fits in a 2^{n-1}:1 MUX with the remaining variable on the data lines (a favourite implementation question). A **demux** does the reverse; a **decoder** (n-to-2^n) activates one output per input combination - a decoder + OR gates implements any SOP function. An **encoder** compresses 2^n lines to n bits; a **priority encoder** resolves simultaneous inputs. A **magnitude comparator** outputs A>B, A=B, A<B.

Exam pointers: "implement f(a,b,c) using an 8:1 MUX" and "using a 4:1 MUX" are repeated numericals - practise the truth-table-to-data-lines mapping. Half/full adder truth tables with K-map derivations, and decoder-based function implementation, are standard theory.

Extra note - Barrel Shifter & ALU - made simple:

Barrel shifter: a circuit that shifts data bits left or right by **any number of positions in a single clock cycle**.

> *Like musical chairs - everyone gets up at once and sits 2 seats ahead. The whole shift happens in one round. A normal shift register moves only 1 seat per tick; a barrel shifter moves 2, 3, 4... whatever you ask, in one tick.*

How it is built: from rows of MUXes (multiplexers). Each output bit has a MUX that chooses which input bit lands there. The shift amount (how far to shift) drives the select lines of the MUXes.

ALU (Arithmetic Logic Unit): the part of a processor that does both **calculation** (add, subtract) and **logic** (AND, OR, XOR).

- Select lines decide the operation: 00 = ADD, 01 = SUB, 10 = AND, 11 = OR (example).
- Inside: full adders + logic gates + a MUX that picks the final answer.

Exam tip: "ALU = adder + logic gates + MUX" - write these 3 things with a diagram and the marks are yours.

Unit 3 - Sequential Circuits

What this unit covers: sequential circuits - circuits with memory: latches, flip-flops, registers, counters, and state-machine design.

Key ideas: Sequential outputs depend on inputs *and* the stored state. A **latch** is level-sensitive, a **flip-flop** edge-triggered. Know the four characteristic tables and equations: **SR** ($Q^+_1 = S + R'Q$), **JK** ($Q^+_1 = JQ' + K'Q$ - no invalid state), **D** ($Q^+_1 = D$), **T** ($Q^+_1 = T⊕Q$). Converting one flip-flop to another uses excitation tables. **Registers** store words; shift registers move bits (SISO, SIPO, PISO, PIPO). **Counters**: asynchronous (ripple - simple but slow due to propagation delay) vs synchronous (all flip-flops share the clock - faster). Design steps for a synchronous counter: state diagram → state table → excitation table → K-maps → circuit. A mod-N counter counts N states.

Exam pointers: "design a mod-6 synchronous counter using JK flip-flops" style questions carry the big marks - practise the full design sequence once, slowly. Characteristic tables/equations of the four flip-flops and the ripple vs synchronous comparison are guaranteed theory.

Extra note - State Reduction & ASM Charts - simple notes:

Why reduce states? Two states are **equivalent** when, for every input, they give the same output and move

to equivalent next-states. Merging them gives a smaller circuit (fewer flip-flops, less hardware) that behaves identically.

State reduction - steps (row-matching / implication table):

1. Write the full state table (present state, next state for each input, output).
2. Find pairs of states with identical outputs and identical (or equivalent) next-states for every input.
3. Merge each equivalent pair into one state and rewrite the table. Repeat until no pair merges.
4. Result: the **minimal state table** - same behaviour, fewest states.

> *Like finding two apps on your phone that do the exact same job - delete one, nothing changes for you, but the phone runs lighter.*

ASM chart (Algorithmic State Machine): a flowchart-style way to describe a sequential circuit. Three blocks: **state box** (one state, lists Moore outputs), **decision box** (diamond - checks an input condition, branches), **conditional output box** (oval - Mealy outputs that depend on state + input). Each path through state box -> decisions -> next state is one clock cycle. To design: draw the ASM chart, convert each state box to a state, read transitions and outputs straight into the state table.

Exam tip: for reduction, always show the equivalence pairs you merged; for ASM, label every box type - examiners deduct for missing state-box names.

Extra note - Pulse Train & PRBS Generator - made simple:

Pulse train generator = a circuit that outputs a **fixed pattern** on every clock tick, again and again. Example: 1, 0, 1, 1, 0 ... then back to 1, 0, 1, 1, 0 ... repeat.

> *Imagine people standing in a line. On every "tick", everyone steps forward and the person at the end of the line steps out. The pattern that comes out is the pulse train.*

How to build it: a shift register (a line of flip-flops) plus a feedback wire that decides what goes back into the start of the line.

Small example: want the pattern `101` (length 3). Take 3 flip-flops: Q1 Q2 Q3. On every tick all bits shift forward and the feedback puts the right new bit at the start. The pattern repeats forever: 101, 101, 101...

Clock generation: a T flip-flop **halves** the input frequency. This is how a slow, steady clock is made from a crystal oscillator.

PRBS (Pseudo-Random Binary Sequence): a magic box that gives "random-looking" bits - but actually from a fixed rule, so it is **pseudo (fake) random**.

How it is built: some middle outputs of the shift register are **XOR**ed and fed back. This is called an **LFSR (Linear Feedback Shift Register)**. With n flip-flops you can get a maximum-length sequence of `2^n - 1` bits before it repeats.

Where used: testing communication links, noise generation, bit-error-rate testing.

Exam tip: draw the shift register, mark the XOR feedback taps, and trace 4-5 clock cycles in a table - that is the standard full-marks answer.

Unit 4 - Logic Families & Semiconductor Memories

What this unit covers: logic families (how real gates are built) and semiconductor memories.

Key ideas: Logic families: **TTL** (transistor-transistor logic - fast, bipolar), **ECL** (emitter-coupled - fastest, high power), **MOS/CMOS** (lowest power, high density, slower than TTL). Compare them on speed, power dissipation, fan-in/fan-out, noise margin - a comparison table question is common. Key parameters: **propagation delay**, **power dissipation**, **fan-out** (how many inputs one output can drive), **noise margin**. Memories: **RAM** (volatile - SRAM is fast flip-flop based, DRAM is capacitor based, denser, needs refresh), **ROM** (non-volatile - PROM, EPROM, EEPROM differ in how they are programmed/erased). Memory capacity questions: a chip with m address lines and n data lines stores 2^k x n bits.

Exam pointers: TTL vs CMOS comparison and RAM vs ROM (with SRAM/DRAM and PROM/EPROM/EEPROM sub-types) are the classic 5-7 mark theory questions. Practise one memory-size numerical (address lines → capacity).

Unit 5 - VLSI Design Flow & VHDL

What this unit covers: how VLSI chips are designed, and VHDL - the language used to describe digital hardware.

Key ideas: The **VLSI design flow**: specification → design entry (VHDL/schematic) → functional simulation → synthesis → place and route → timing verification → fabrication. Know the difference between simulation levels (behavioural, RTL, gate). **VHDL** basics: every design has an **entity** (ports: in, out, inout) and an **architecture** (the internal logic). Modelling styles: **dataflow** (concurrent signal assignments), **behavioural** (process with if/case - sequential statements), **structural** (connecting components like a netlist). Learn the skeleton of one example each: a 2:1 MUX, a full adder, a D flip-flop.

Exam pointers: "write VHDL code for a full adder / 2:1 MUX / D flip-flop, any modelling style" is the standard coding question - memorise the entity-architecture skeleton once and reuse it. Theory: design-flow diagram

and the three modelling styles with one-line differences.